GeekUp

PRODUCT BACKEND ENGINEER TEST

Concert Ticket Booking Platform Backend System

Đinh Cao Thiên Lộc - loc.dinhbh@hcmut.edu.vn

Ho Chi Minh City, May 2026

1 Requirements

1.1 Functional Requirements

1.1.1 Customer-Facing Booking Flows

- Browse available concerts.
- View ticket categories and prices.
- Reserve tickets.
- Apply promotional vouchers during the booking process.
- Track the status of their bookings.

1.1.2 Internal Operation Dashboard

- Monitor all active bookings.
- Manage and publish tickets for new concerts.
- Validate ticket availability.
- Manage voucher campaigns.
- Handle failed or suspicious booking transactions.
- Manually update booking status when necessary.

1.2 Non-Functional Requirements

1.2.1 Performance and Scalability

- The system must support approximately 50,000 users during launch week.
- The system must handle peak traffic of 300–500 booking requests per minute.
- The architecture should scale horizontally when traffic increases.

1.2.2 Concurrency and Data Integrity

- Prevent overselling of limited concert tickets.
- Prevent duplicate bookings caused by retries or concurrent requests.
- Ensure promotional vouchers cannot be abused or reused improperly.
- Maintain transactional consistency during flash sale events.

1.2.3 System Stability

- The platform must remain stable during flash sale traffic spikes.
- The system should recover gracefully from temporary failures.
- APIs should include proper timeout and retry handling mechanisms.

1.2.4 Documentation and Maintainability

- Provide documentation for system architecture and database design.
- Provide setup instructions for local development and testing.
- Deliver API documentation using tools such as Swagger.
- Provide API testing collections using tools such as Postman.
- Clearly document assumptions, limitations, and supported features.

2 Data Model

2.1 Database Architecture

This database schema is designed as a highly normalized relational model tailored specifically for a high-traffic **Concert Ticket Booking Platform**. The architecture strictly separates core entities such as **Users**, **Concerts**, **TicketCategories**, **Vouchers**, and **Bookings** to maintain maximum **data integrity** and minimize **data redundancy**.

2.2 Special Applications for Flash Sale Operations

Handling a flash sale requires the database to operate as the **single source of truth** in order to prevent critical business failures such as **overselling inventory** or **duplicate payments**. The schema integrates multiple mechanisms to guarantee **stability**, **consistency**, and **accuracy** during extreme traffic spikes (e.g., **500 booking requests per minute**).

2.2.1 Concurrency Control to Prevent Overselling

During a flash sale, thousands of users may attempt to purchase the exact same ticket simultaneously. To solve this **race condition**, the system utilizes a **row-level locking mechanism** using **Pessimistic Locking** on the **TicketCategories** table.

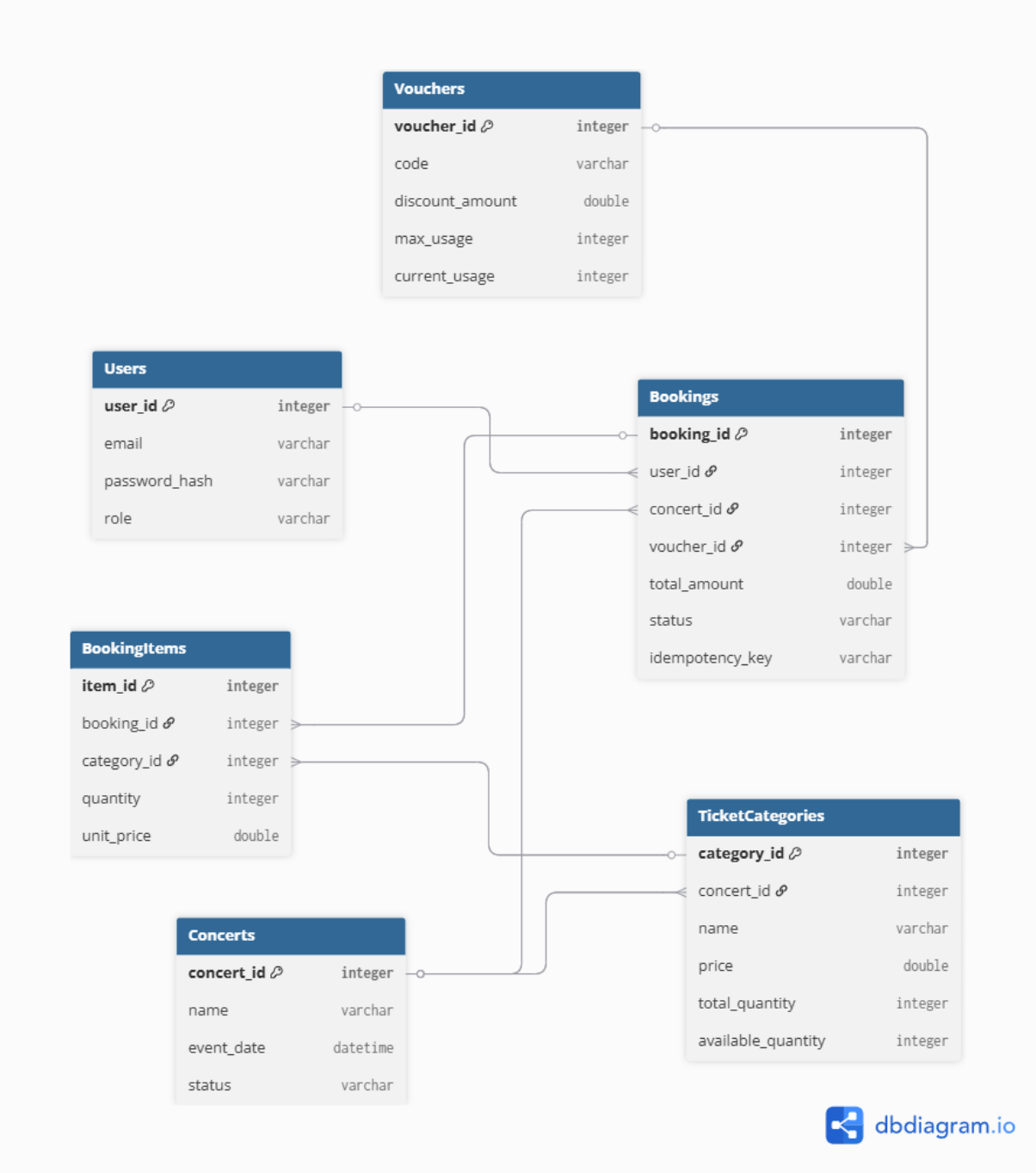
When a booking transaction begins, the database executes a locking query using:

```
SELECT ... FOR UPDATE
```

This operation temporarily locks the selected ticket row, forcing concurrent transactions to wait until the current transaction either commits or rolls back. As a result, the system guarantees that only one transaction can verify and deduct the **available_quantity** at a time, effectively preventing **overselling** during high concurrency situations.

2.2.2 Idempotency Key against Duplicate Bookings

During high traffic periods, **network latency**, **timeouts**, or repeated user interactions (e.g., double-clicking the checkout button) may generate multiple identical booking requests.

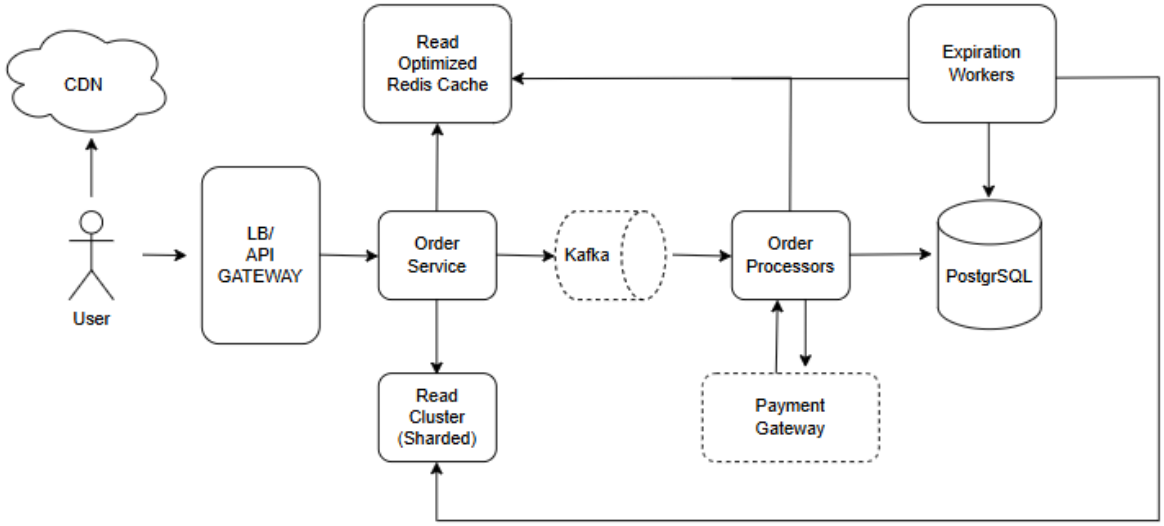


Hình 1: database diagram

To prevent **duplicate bookings**, the **idempotency_key** column in the **Bookings** table is defined with a **UNIQUE constraint**. This key is generated by the frontend client for every booking attempt.

If the same request is accidentally submitted multiple times, the database automatically rejects subsequent inserts containing the same idempotency key. This mechanism guarantees that a booking action is processed **exactly once**, ensuring transaction safety and preventing users from being charged multiple times for the same purchase.

3 System Architecture Components



Hình 2: System Architecture diagram

3.1 Content Delivery Network (CDN)

The **Content Delivery Network (CDN)** absorbs nearly **99.9%** of the pre-sale read traffic. It serves cached static assets such as **UI components**, **images**, and synchronizes the **product countdown timer** locally within the client's browser.

This significantly reduces the number of requests reaching the origin backend servers and prevents infrastructure overload during flash sale events.

3.2 API Gateway and Load Balancer

The **API Gateway** acts as the first line of defense against the **"thundering herd"** problem during sudden traffic spikes.

Its responsibilities include:

- Filtering malicious bot traffic.
- Enforcing strict **rate limiting** using algorithms such as the **Token Bucket Algorithm**.
- Restricting users to approximately **1 request per second**.
- Evenly distributing legitimate traffic across multiple stateless **Order Service** nodes.

3.3 Main API Server (Order Service)

The **Order Service** receives incoming checkout requests and utilizes **deterministic routing** through **consistent hashing** based on the **User ID**. This guarantees that requests are routed to the correct **Redis shard**.

The service performs the following operations:

- Triggers an atomic **Lua Script** inside Redis for immediate inventory deduction.
- Publishes successful reservation events into **Apache Kafka**.
- Returns an asynchronous **HTTP 202 Accepted** response together with a **Queue Ticket ID**.
- Instructs the frontend application to begin **short polling**.

This architecture decouples the user experience from slower database persistence operations.

3.4 Write-Optimized Redis Cluster (Inventory Manager)

The **Write-Optimized Redis Cluster** stores pre-allocated ticket inventory across multiple shards to avoid the **"hotkey"** problem and single-core CPU bottlenecks.

Redis executes atomic **single-threaded Lua Scripts** to:

- Verify whether **stock** $>$ **0**.
- Decrease the **available_quantity** safely in microseconds.
- Prevent race conditions during high concurrency.

Additionally, the system applies a strict **5-minute Time-To-Live (TTL)** temporary reservation hold for every successful ticket reservation.

3.5 Apache Kafka (Message Broker)

Apache Kafka acts as a highly durable **message broker** and system **"shock absorber"** between the high-speed caching layer and the slower persistence layer.

Kafka safely stores checkout events on disk and provides:

- High durability.
- Zero data loss guarantees.
- Controlled database ingestion throughput.
- Asynchronous communication between services.

This design ensures the database remains stable even during massive traffic spikes.

3.6 Background Order Processor (Worker Node)

The **Worker Node** asynchronously consumes checkout events from Kafka at a controlled rate.

Its responsibilities include:

- Persisting official **PENDING** booking records into PostgreSQL.
- Communicating with external payment gateways such as **VNPay** or **Stripe**.
- Generating secure payment URLs.
- Pushing payment URLs into the **Read-Optimized Redis Cache**.

This asynchronous workflow prevents the checkout API from blocking users while waiting for payment provider responses.

3.7 Primary Relational Database (PostgreSQL)

PostgreSQL serves as the ultimate **system of record** and the single source of truth for all persistent transactional data.

The database guarantees:

- Full **ACID compliance**.
- Strong transactional consistency.
- Referential integrity between normalized tables.
- Enforcement of **UNIQUE constraints**, especially for the **Idempotency Key**.

Core relational tables include:

- Users
- Concerts
- TicketCategories
- Bookings
- BookingItems

3.8 Read-Optimized Redis Cache

The **Read-Optimized Redis Cache** separates heavy read traffic from critical write-heavy inventory operations by following the **CQRS (Command Query Responsibility Segregation)** pattern.

Its main responsibilities include:

- Handling continuous frontend **short polling** requests.

- Delivering payment URLs instantly once available.
- Minimizing unnecessary load on PostgreSQL.

The frontend periodically queries Redis every few seconds until the payment URL becomes available.

3.9 Expiration Worker (Cleanup Sweeper)

The **Expiration Worker** periodically scans for expired **PENDING** orders whose 5-minute payment window has elapsed.

Its recovery responsibilities include:

- Cancelling expired booking records in PostgreSQL.
- Removing payment URLs from Redis cache.
- Returning abandoned inventory back into the original Redis shard.
- Restoring ticket availability to the public inventory pool.

This mechanism guarantees automatic inventory recovery and prevents ticket leakage caused by abandoned checkout sessions.

3.10 High-Level Checkout Flow (Flash Sale)

1. **CDN Offloading:** A Content Delivery Network (CDN) absorbs 99.9% of pre-sale read traffic by serving cached product details and a server-synced countdown timer to millions of waiting users.
2. **Client-Side Trigger:** Client-side JavaScript manages the local countdown and visually reveals the "Buy" button on the user's screen at the exact second the sale begins.
3. **API Gateway Defense:** The API Gateway acts as the first line of defense by filtering bots, validating session tokens, and enforcing strict rate limits (e.g., 1 request per second).
4. **Deterministic Routing:** The stateless Order Service receives clean traffic, hashes the User ID, and deterministically routes the request to a specific shard within the Redis cluster using consistent hashing.
5. **Atomic Lua Execution:** An atomic Lua script executes on the targeted Redis shard to check and decrement inventory in a single indivisible step, placing a 5-minute TTL hold on the item.
6. **Asynchronous Decoupling:** On successful Redis decrement, the Order Service publishes a *checkout event* to Kafka and immediately returns `202 Accepted` with a Queue Ticket ID to the client.
7. **Client Short Polling:** The client UI displays a loading spinner and begins short polling a separate read-optimized Redis cache for its payment link.
8. **Order Processing:** Worker nodes (Order Processors) drain the Kafka queue at a steady, controlled rate and insert initial *Pending Payment* records into PostgreSQL.

9. **Payment URL Generation:** In parallel, workers call an external Payment Gateway (e.g., Stripe) to generate a secure checkout URL, then persist it to PostgreSQL and push it to the read-optimized Redis cache.
10. **Browser Redirect:** The polling request eventually retrieves the payment URL from cache, triggering an automatic redirect to the external payment page.
11. **Order Finalization:** After payment completion, the Payment Gateway sends an asynchronous webhook to the system, which updates the PostgreSQL order status to *Confirmed*.
12. **Timeout Cleanup:** An Expiration Worker monitors the 5-minute window; if payment is not completed, it cancels the database record and atomically increments inventory back on the original Redis shard.

4 Solutions for building a high-concurrency Flash Sale system

4.1 Preventing Overselling

To prevent overselling, the system moves inventory management from a slow relational database to an in-memory Redis cluster.

- Using Atomic Lua Scripts to ensure that checking stock and decrementing it happens as a **single indivisible operation**, preventing race conditions.
- Implementing Inventory Sharding, where the total tickets are split across multiple Redis nodes to avoid CPU bottlenecks and users are routed to specific shards using Consistent Hashing.

4.2 Surviving the Thundering Herd

Standard autoscaling is too slow for flash sales because traffic spikes from zero to hundred in seconds.

- The solution is Infrastructure Pre-warming, where servers are manually over-provisioned hours before the sale.
- Additionally, Multi-tier Rate Limiting (at the API Gateway) filters out bots and malicious actors.
- Message Queue (Kafka) acts as a "shock absorber,"allowing the system to ingest tens of thousands of requests per second while the database processes them at a slower, sustainable rate.

4.3 Distributed Transactions Inventory Holds

Since inventory is reserved in Redis but finalized in PostgreSQL after payment, the system uses a Reservation Hold with a strict TTL. (e.g., 5 minutes)

- Applying the Saga Pattern to manage consistency across services without heavy ACID locks.
- If a user fails to pay, a Background Sweeper (Expiration Worker) identifies the expired hold, cancels the order in the database, and atomically increments the inventory back to the correct Redis shard.

4.4 Polling vs Websockets

The system rejects Websockets because maintaining hundred of persistent connections is operationally expensive and unstable during a massive spike.

- It uses Short Polling, where the client sends a lightweight GET request every 2 seconds to check the order status.
- To prevent this polling from overwhelming the main database, the worker nodes write the payment URL to a Read-Optimized Redis Cache, allowing the polling requests to be answered with sub-millisecond latency.

5 Assumptions / What Was Done / What Was Not Done

5.1 Assumptions

- The project follows a layered API architecture: **Controller →Service →Repository**.
- External infrastructure is expected to run locally via Docker Compose: **PostgreSQL, Kafka (with Zookeeper), and Redis**.
- API contracts use DTOs: `dto/request` for request bodies and `dto/response` for response bodies.

5.2 What Was Done

- Created a root documentation file `.readme` containing: coding guidelines/conventions, how to add a new feature, unit testing notes (JUnit 5 + Mockito), and local setup/run steps.
- Integrated Swagger UI by adding the dependency `springdoc-openapi-starter-webmvc-ui` to `pom.xml`.
- Added Postman artifacts under `/docs/postman`: a Collection JSON (Checkout, Order Status, Inventory Check) and an Environment JSON with `base_url = http://localhost:8080`.
- Implemented a minimal Inventory API endpoint to make the Postman *Inventory Check* request usable: `GET /api/v1/inventory/{categoryId}`. The endpoint reads remaining stock from Redis (key `ticket_stock:{categoryId}`) and falls back to the database `totalQuantity` if the Redis key is missing.
- Verified the build by running `mvn test` successfully.

5.3 What Was Not Done (Limitations)

- No new business workflow (e.g., a full order lifecycle beyond the existing booking flow) was designed or implemented.
- No authentication/authorization, rate limiting, or API versioning policy was added.
- No additional automated tests were introduced beyond ensuring the current test suite still passes.